

Добрый день! (меня слышно?) Тогда начнем. ЗАСЕЧЬ ТАЙМЕР

Меня зовут Александр Дупак, и я хотел бы рассказать о своем решении задачи полуфинала – построения регулятора для движения квадрокоптера по заранее неизвестной траектории.

Задача была поставлена следующим образом. Необходимо написать алгоритм для движения квадрокоптера по трассе, ширина которой составляет 8м и на которой нанесена линия толщиной 5см. Высота может быть произвольно заданной (до 20м). На полет дается 5 минут, за которые необходимо собрать наибольшее количество виртуальных монеток, расположенных вдоль трассы. Монетки имеют неизвестный радиус, расстояние между монетками также не регламентировано. Таким образом можно обобщить критерии алгоритма как точность следования траектории (нужно обеспечить минимальное отклонение от нанесенной линии) и скорость, поскольку нужно за ограниченное время собрать максимальное количество монеток.

Рассмотрим наиболее распространенные подходы для решения этой задачи. Прежде всего сердцевина алгоритма — это регулятор, который сможет обеспечить качественное слежение за траекторией. Несмотря на, казалось бы, очевидную задачу существует огромное количество различных подходов для ее решения. Прежде всего это всем известный ПИД-регулятор, для линейных систем наиболее распространен LQR-регулятор который также может быть применен к квадрокоптеру. Также возможно построение регулятора на основе метода функции Ляпунова. Еще существуют адаптивные регуляторы такие как МРС, а в последнее время вообще стало актуально синтезировать регулятор для системы рассматривая ее как черную коробку, которая имеет вход или несколько входов и один или несколько выходов. И анализируя зависимость выхода системы от ее входа можно построить регрессионную модель системы и на ее основании уже синтезировать регулятор. Не имея знаний о физических процессах, которые протекают внутри системы. Но обо всем по отдельности. Рассмотрим некоторые из них в отдельности.

Я хотел бы сравнить данные регуляторы в формате плюсы/минусы. И начнем с ПИД-регулятора. Основной плюс — это простота реализации, но обратной стороной является ручная подстройка коэффициентов. Есть некоторые методы для их расчета: с помощью полинома Ньютона или Баттерворта или по методу Зиглера-Николаса, но на практике подбор коэффициентов работает лучше, что в некоторых случаях просто недопустимо. Не будем долго задерживать на ПИД-регуляторе. Перейдем к такому регулятору как LQR –

линейно квадратичный регулятор. Он основан на принципе минимизации критерия качества, который определяется матрицами Q (штрафы), R (веса), N накладывающих ограничения на соответствующие координаты системы.

Результатом расчета является матрица K оптимальных обратных связей по переменным состояния. Стоит отметить, что этот регулятор работает только для линейных или линеаризованных систем, и предполагает полную идентификацию модели, что в нашем случае сделать проблематично. Один из подходов который этого не требует это синтез регулятора с использованием метода функций Ляпунова. В отличие от других регуляторов он позволяет исключить увеличение ошибки регулирования (т.е. отдаление от желаемой цели из-за перерегулирования например). Его принцип заключается в следующем. Выбирается функция Ляпунова такая что ее производная будет отрицательна и тогда вектора градиента функции Ляпунова будут противонаправлены векторам поля дифференциального уравнения, таким образом он обеспечивает стремление траекторий к точке равновесия системы. Минусом данного подхода является то что вблизи точки равновесия скорость получается очень незначительной и даже в некоторых случаях недостаточной чтобы преодолеть внешние силы трения системы, из-за чего образуется статическая ошибка регулирования. Совершенно другой подход предполагает MPC. Он позволяет реализовать адаптивное регулирование на основе оптимизации модели системы на каждой итерации алгоритма. Позволяет добиться хороших результатов на высоких скоростях. Минусом является более сложная реализация как программная, так и аппаратная, поскольку он требует большие вычислительные мощности.

Перейдем к моему решению. Я решил воспользоваться робастным регулятором, который, во-первых, позволяет избавиться от программного дифференцирования, во-вторых, предоставляет возможности для подстройки параметров, которые зависят от итерационного шага программы. А также позволяет добиться в каком-то смысле качественного регулирования по независимой переменной x .

Я хотел бы продемонстрировать сравнение этого регулятора с обычным ПИД регулятором. Стоит обратить внимание на то, что ПИД-регулятор имеет как бы заносы на поворотах, но очень хорошо придерживается референсной точки на прямых. В то время как робастный регулятор, наоборот, на прямых имеет большее отставание, зато на поворотах имеет такую способность как бы срезать углы. То есть при должном подборе коэффициентов он по-хорошему должен проходить повороты по внутреннему радиусу, в то время как ПИД-регулятор такого не умеет и в любом случае в меньшей или большей степени

отклоняется по внешнему радиусу поворота. Я не подбирал параметры чтобы регуляторы следовали идеально, иначе было бы не видно их различия в принципе работы.

Я не хочу утверждать, что один регулятор однозначно лучше, а другой хуже, а лишь хочу показать еще один подход к решению этой задачи.